

Getting started with Amazon CloudFront

AWS Prescriptive Guidance

Amazon CloudFront is a fast content delivery network (CDN) service that securely delivers data, videos, applications, and APIs to customers globally with low latency and high transfer speeds. This guide covers CloudFront distributions, origins, cache behaviors, caching strategies, security features including WAF and Shield, Lambda@Edge, CloudFront Functions, real-time logging, and optimization best practices for building high-performance global web applications.

Authors: Wei Chen and Isabella Rossi, Amazon Web Services (AWS)

Date: January 2026

Table of Contents

- [Introduction](#)
- [How CloudFront Works](#)
- [Distributions and Origins](#)
- [Cache Behaviors and Caching](#)
- [Security](#)
- [Lambda@Edge and CloudFront Functions](#)
- [Real-Time Logging](#)
- [Pricing](#)
- [Best Practices](#)
- [FAQ](#)

Introduction

Amazon CloudFront is a global content delivery network (CDN) that accelerates the delivery of websites, APIs, video streams, and other web assets to users worldwide. CloudFront routes each user request to the edge location that provides the lowest latency — often within tens of milliseconds. CloudFront has over 600 Points of Presence (PoPs) across 100+ cities in 50+ countries, providing massive global reach.

CloudFront integrates deeply with AWS services, including Amazon S3, Application Load Balancer, AWS API Gateway, AWS Lambda, AWS WAF, AWS Shield Advanced, and AWS Certificate Manager. This integration allows you to build sophisticated, globally distributed web applications with built-in security and performance optimization.

How CloudFront Works

When a user requests content served through CloudFront, the request is routed to the nearest CloudFront edge location using Anycast routing. The edge location checks its cache for the requested object. If the object is in cache and has not expired (a cache hit), CloudFront immediately delivers it to the user. If the object is not in cache or has expired (a cache miss), CloudFront fetches it from the configured origin server, caches it for future requests, and delivers it to the user.

CloudFront also uses a network of Regional Edge Caches (RECs) between your origin and the edge locations. RECs hold more content than individual edge locations. When an edge location experiences a cache miss, it first checks the nearest REC before going to the origin. This second-layer caching further reduces the load on your origin and improves hit rates.

Distributions and Origins

A CloudFront distribution is your configuration for delivering content. When you create a distribution, you specify at least one origin — the source of the content that CloudFront caches and delivers. An origin can be an Amazon S3 bucket, an Application Load Balancer, an Amazon API Gateway endpoint, an AWS Elemental MediaStore container, or any custom HTTP server.

CloudFront supports multiple origins per distribution with origin groups for automatic failover. If the primary origin is unavailable or returns a specific HTTP status code (4xx or 5xx), CloudFront automatically routes requests to the secondary origin in the group, providing high availability for your content.

Origin Access Control (OAC)

Origin Access Control (OAC) is the recommended way to restrict access to your S3 origin so that only CloudFront can retrieve objects directly from the bucket. OAC supports all S3 buckets in all Regions, server-side encryption with AWS KMS, dynamic PUT and DELETE requests, and all S3 bucket types including S3 on Outposts.

Cache Behaviors and Caching

Cache behaviors define how CloudFront processes requests for different URL path patterns. A distribution must have at least one default cache behavior (*), and you can add up to 25 additional cache behaviors. Cache behaviors control which origin to forward requests to, whether to cache based on query strings or headers, the cache TTL, and whether to enforce HTTPS.

CloudFront uses cache policies to define what CloudFront uses as the cache key and the origin request policy to define what CloudFront forwards to the origin. Separating the cache key from the origin request allows you to cache at the right granularity without unnecessarily forwarding information to the origin.

Cache TTL Settings

Object Type	Recommended Cache-Control	Minimum TTL	Maximum TTL
Static assets (CSS, JS, images)	max-age=31536000, immutable	86400 (1 day)	31536000 (1 year)
HTML pages	no-cache or max-age=0	0	0
API responses	no-store or max-age=60	0	300
Videos/large media	max-age=86400	3600	604800
Error pages	Custom	10	300

Security

HTTPS and TLS

CloudFront supports HTTPS for communication between viewers and CloudFront, and between CloudFront and your origin. You can use AWS Certificate Manager (ACM) to provision free SSL/TLS certificates for your custom domain names. CloudFront supports TLS 1.2 and 1.3 and multiple security policies (cipher suites) for viewer connections. Always configure CloudFront to redirect HTTP to HTTPS.

AWS WAF Integration

AWS WAF (Web Application Firewall) integrates directly with CloudFront to inspect web requests before they reach your origin. You can use WAF to allow or block requests based on IP addresses, geographic location, rate limits, SQL injection patterns, cross-site scripting, and custom rules. AWS Managed Rule Groups provide pre-configured protection against common threats including the OWASP Top 10.

AWS Shield

All CloudFront distributions are protected by AWS Shield Standard at no extra charge. Shield Standard provides automatic protection against the most common and frequent DDoS attacks. AWS Shield Advanced provides additional protections for CloudFront distributions, including near-real-time visibility into attacks, DDoS cost protection, and access to the AWS DDoS Response Team (DRT).

Geo Restriction

CloudFront geographic restriction (also called geo-blocking) allows you to prevent users in specific countries from accessing your content. You can create an allowlist of countries that can access your content, or a blocklist of countries that are denied access. Geographic restriction applies to an entire distribution.

Lambda@Edge and CloudFront Functions

Lambda@Edge lets you run Node.js or Python Lambda functions at CloudFront edge locations in response to CloudFront events. There are four event trigger points: viewer request (when CloudFront receives a request from a viewer), origin request (before CloudFront forwards a request to the origin), origin response (after CloudFront receives a response from the origin), and viewer response (before CloudFront returns the response to the viewer).

CloudFront Functions are lightweight JavaScript functions that execute at CloudFront edge locations with sub-millisecond startup times and low cost. They are ideal for simple request/response manipulation, URL rewrites and redirects, header manipulation, and access token validation. CloudFront Functions execute at all 600+ PoPs, while Lambda@Edge executes at Regional Edge Caches.

Best Practices

- Use versioned filenames or cache busting for static assets: Embed content hashes in filenames (e.g., main.abc123.js) for aggressive, long-lived caching with instant invalidation via filename change.
- Enable compression: Configure CloudFront to automatically compress text-based objects (HTML, CSS, JS, JSON) using gzip or Brotli, reducing transfer size by 60–80%.
- Use AWS WAF with Managed Rule Groups: Protect against common web exploits at the edge before traffic reaches your origin.
- Monitor with CloudWatch and Real-Time Logs: Track CacheHitRate, TotalErrorRate, and Requests metrics. A low cache hit rate indicates suboptimal caching configuration.
- Invalidate cache sparingly: S3 versioning and cache-busting filenames are more efficient than manual CloudFront invalidations, which are slower and may incur costs.

- Use origin groups for high availability: Configure origin failover to ensure content delivery even if the primary origin experiences issues.
- Restrict direct origin access: Use Origin Access Control (for S3) or custom headers (for custom origins) to ensure all traffic flows through CloudFront.
- Use CloudFront Functions for simple edge logic: Prefer CloudFront Functions over Lambda@Edge for lightweight operations that need to run at every PoP.

Frequently Asked Questions

Q: How do I invalidate cached content in CloudFront?

A: You can create an invalidation request in the CloudFront console, CLI, or API, specifying the paths to invalidate. Wildcards are supported. The first 1,000 invalidation paths per month are free; additional paths are charged. Using versioned filenames is the preferred alternative.

Q: Can CloudFront serve dynamic content?

A: Yes. CloudFront can cache and deliver dynamic content, including API responses, with appropriate cache policies. For fully dynamic, uncacheable content, CloudFront still benefits origin performance by maintaining persistent TCP connections with the origin and using the AWS backbone network.